Temporary Secure Sharing Platform

Project Name

Ephemeral Cross-Device Sharing Platform

Objective:

Build a secure, lightweight platform that allows users to temporarily share files and text-based content between devices anywhere in the world using a short access code. Users can create either a Quick Transfer session or a Shared Workspace session. Every session remains available for exactly five minutes and is automatically deleted after expiration.

Scope of Work:

- **Session Creation:** Allow users to create either a Quick Transfer or Shared Workspace session and generate a secure, short-lived access code.
- **Quick Transfer:** A user uploads content and shares the generated code. Anyone with the valid code can access the content during the five-minute session.
- **Shared Workspace:** Multiple users can join the same session using the code and collaboratively share content during the session, subject to a configurable participant limit.
- **Content Types:** Support files such as documents, images, audio, video and ZIP folders, along with text-based content such as notes, code snippets, clipboard content and URLs.
- **Automatic Expiration:** Sessions and their associated content must become inaccessible and be deleted automatically after five minutes.
- **Real-Time Updates:** Shared Workspace participants should receive newly added content without manually refreshing the page.

Input

- Session type: Quick Transfer or Shared Workspace.
- Files and binary content: documents, images, audio, video, ZIP folders and other supported file types.
- Text content: notes, clipboard content, code snippets and URLs.
- Optional session configuration such as maximum participants.

Output

- A unique short access code for the created session.
- A temporary session interface showing available shared content.
- Secure access to files and text for authorized session participants.
- Real-time content updates in Shared Workspace sessions.
- Automatic removal and invalidation of the session and its content after five minutes.

Technical Expectations:

- Frontend application providing session creation, code-based joining, uploads, downloads and content display.
- Backend REST APIs for session and content management.
- WebSocket-based real-time communication for Shared Workspace sessions.
- PostgreSQL or equivalent relational database for session and content metadata.
- Redis or equivalent in-memory store for short-lived session data, participant state and expiration management.
- Object storage such as Amazon S3, Cloudflare R2 or MinIO for binary files.
- Secure random session codes with protection against code guessing and unauthorized access.
- Automatic cleanup of expired sessions, metadata and stored files.
- File validation, size limits, rate limiting and appropriate security controls.
- Containerized deployment using Docker, with production-ready configuration and HTTPS.

Deliverables:

- Fully functional web application implementing both session types.
- Backend API and database implementation.
- Object-storage integration for binary content.
- Real-time Shared Workspace implementation.
- Automatic five-minute session expiration and cleanup mechanism.
- Responsive frontend interface.
- Dockerized application and deployment configuration.
- API documentation and project README.
- System architecture and data-flow documentation.
- Deployed demonstration of the application.

Evaluation Focus:

- **Correctness:** Reliable creation, joining, sharing and expiration of sessions.
- **Architecture:** Appropriate use of databases, caching, object storage and real-time communication.
- **Security:** Secure access codes, authorization, validation, rate limiting and protection against common attacks.
- **Scalability:** Ability to handle multiple concurrent users, large files and multiple active sessions efficiently.
- **Reliability:** Correct cleanup, failure handling, upload/download robustness and consistent session state.
- **User Experience:** Simple code-based workflow, clear countdowns, upload progress and intuitive content sharing.

- **Code Quality:** Maintainable structure, clear APIs, modular implementation, testing and documentation.
- **Deployment:** Reproducible Docker-based setup and a working production deployment.